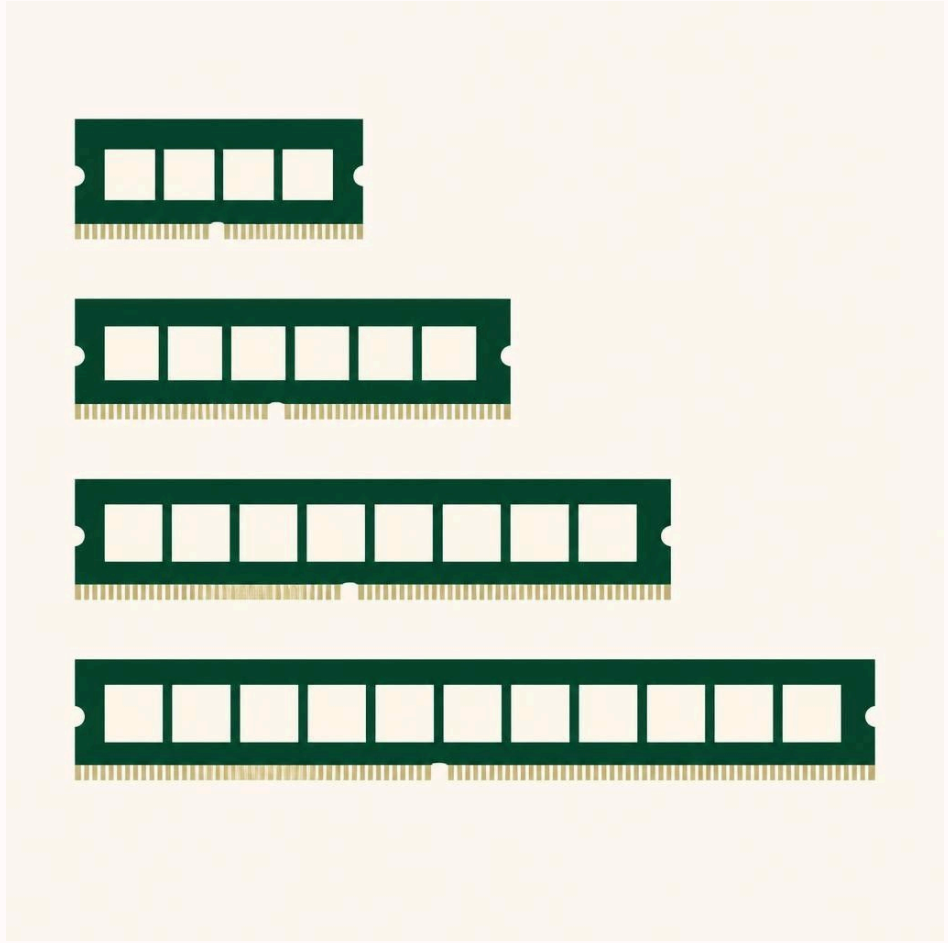
calloc e realloc in C

calloc, realloc, realloc in pratica

LINGUAGGIO C | MEMORIA DINAMICA | CALLOC E REALLOC

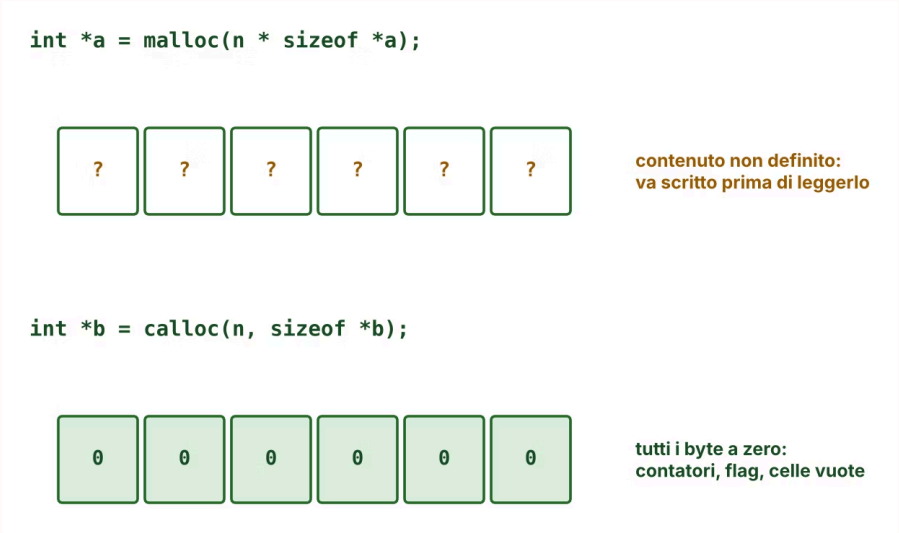


calloc: un Blocco Azzerato

calloc riceve il numero di elementi e la dimensione di ciascuno, e mette a zero tutti i byte del blocco.

```c
int *a = malloc(n * sizeof *a);     /* contenuto non definito */
int *b = calloc(n, sizeof *b);      /* n int che valgono 0 */

if (b == NULL) {
    printf("Memoria insufficiente.\n");
}
...
free(b);
```



> calloc conviene quando il programma parte davvero da zero: contatori, totali, flag, celle vuote. Se ogni elemento verrà scritto prima di essere letto, basta malloc.

Contatori e Matrici Azzerate

```
int *conta = calloc(LETTERE, sizeof *conta);
conta[c - 'a']++;

int *assente = calloc(giorni * studenti, sizeof *assente);
assente[g * studenti + s] = 1;

char *composto = calloc(n + 1, sizeof *composto);
```

	malloc	**calloc**
argomenti	n * sizeof *p	n, sizeof *p
contenuto iniziale	non definito	byte a zero
quando	dati scritti subito	contatori, flag

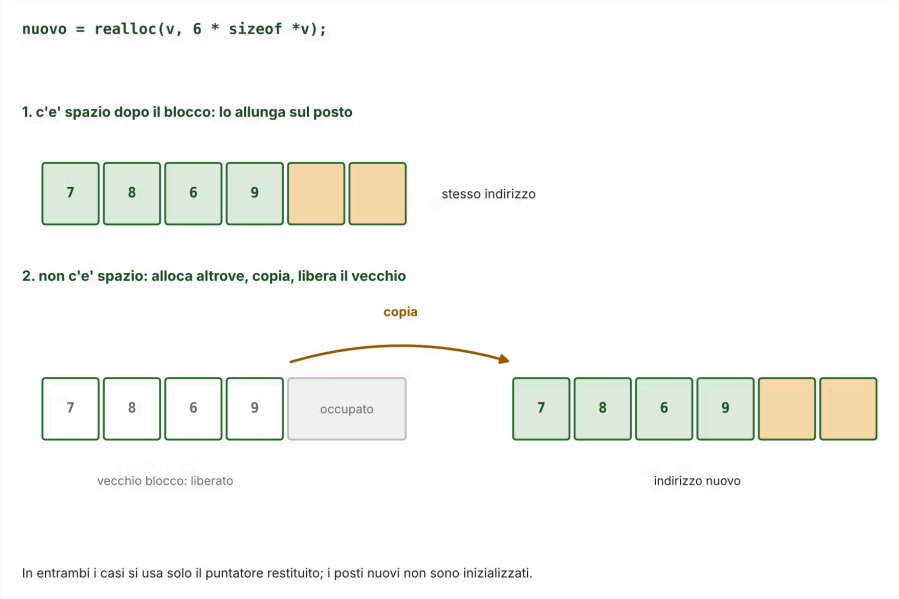
> calloc azzera i byte: per gli interi è 0; per double e puntatori lo standard non lo garantisce, ma su tutte le macchine comuni valgono 0.0 e NULL.

realloc: Cambiare Dimensione

realloc restituisce un blocco della nuova dimensione che contiene i dati del vecchio. Se c'è spazio lo allunga sul posto, altrimenti alloca altrove, copia e libera il vecchio.

```
nuovo = realloc(temperature, 14 * sizeof *nuovo);
if (nuovo != NULL) {
   temperature = nuovo;
   /* i primi 7 valori ci sono ancora,
      gli altri 7 vanno scritti */
}
```

Dopo realloc si usa solo il puntatore restituito: l'indirizzo può essere cambiato. I posti aggiunti non sono inizializzati.

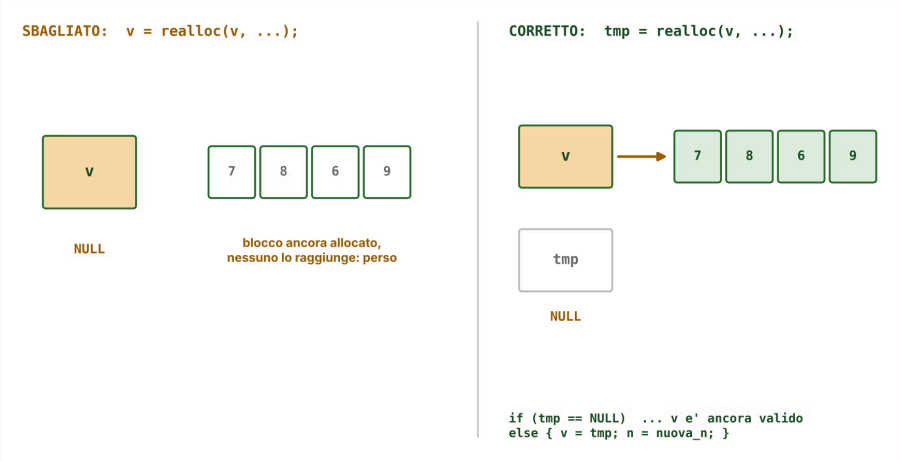


Il Puntatore Temporaneo

Se realloc non trova memoria restituisce NULL e non libera il blocco vecchio, che resta valido con i suoi dati.

```c
v = realloc(v, nuova_n * sizeof *v);    /* SBAGLIATO */

tmp = realloc(v, nuova_n * sizeof *v);  /* corretto */
if (tmp == NULL) {
    /* v è ancora valido */
} else {
    v = tmp;
    n = nuova_n;
}
```



> La dimensione si aggiorna solo dopo che realloc è riuscita.

Ridurre un Blocco

```
punteggi = malloc(MASSIMO * sizeof *punteggi);
... lettura di n punteggi ...

if (n == 0) {
    free(punteggi);
    punteggi = NULL;
} else {
    tmp = realloc(punteggi, n * sizeof *punteggi);
    if (tmp != NULL) {
        punteggi = tmp;
    }
}
```

Quando non si sa quanti dati arriveranno si può allocare un massimo, riempirlo e poi restituire la parte non usata.

Un filtro sul posto, con un indice che legge e uno che scrive, toglie i valori errati senza un secondo array; poi realloc riduce il blocco.

> Mai realloc(p, 0): il risultato dipende dalla libreria. Per svuotare si usano free e NULL.

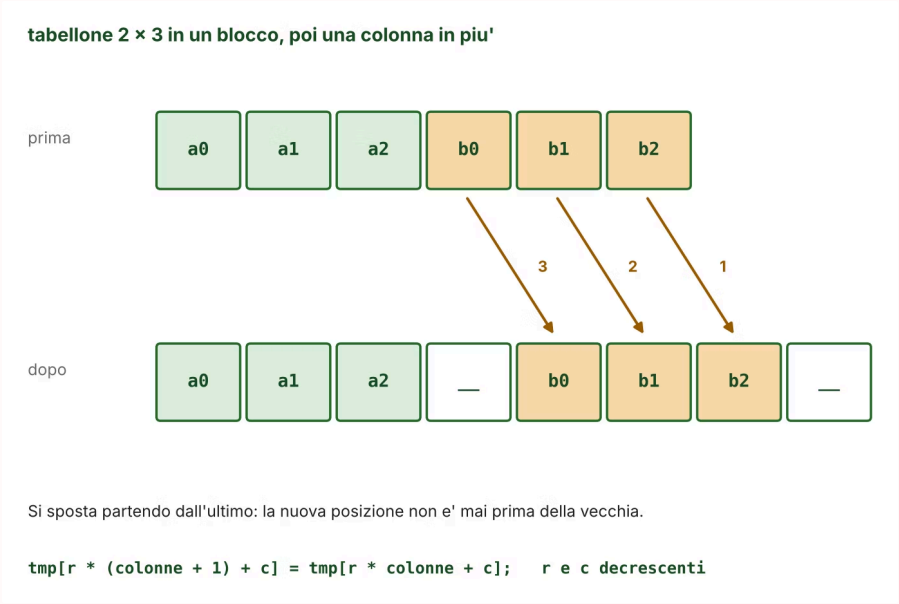
Una Colonna in più

Un tabellone in un solo blocco ha le righe una dopo l'altra: aggiungere una colonna cambia la lunghezza di ogni riga, quindi dopo la realloc i voti si spostano.

```
tmp = realloc(tab, righe * (colonne + 1) * sizeof *tmp);

for (r = righe - 1; r >= 0; r--) {
    for (c = colonne - 1; c >= 0; c--) {
        tmp[r * (colonne + 1) + c] = tmp[r * colonne + c];
    }
}
```

> Si parte dall'ultimo voto: la nuova posizione non è mai prima della vecchia, quindi nessun voto viene sovrascritto prima di essere spostato.



realloc con NULL e Costo della Crescita

```c
char **parole = NULL;
int n = 0;

tmp = realloc(parole, (n + 1) * sizeof *parole);
if (tmp != NULL) {
    parole = tmp;
    parole[n] = copia;
    n++;
}
```

realloc(NULL, byte) funziona come malloc: un elenco può partire vuoto e crescere sempre con realloc.

elementi	crescita	realloc	copie (max)
10 000	+1	10 000	49 995 000
10 000	+16	625	3 120 000

> Crescere di un elemento alla volta costa come n^2: 03-array-dinamici mostra una strategia molto migliore.

Indici, non Puntatori

```
Studente *capoclasse = &classe[2];
tmp = realloc(classe, 5 * sizeof *classe);
capoclasse->voto      /* SBAGLIATO: pendente */

int capoclasse = 2;
tmp = realloc(classe, 5 * sizeof *classe);
classe = tmp;
classe[capoclasse].voto     /* corretto */
```

Se realloc sposta il blocco, ogni puntatore a un suo elemento diventa pendente. Gli indici invece restano validi: l'elemento 2 è sempre classe[2].

Le prenotazioni del laboratorio ricordano la postazione con l'indice: quando arrivano postazioni nuove il blocco si sposta, ma le prenotazioni restano corrette.

> Per ricordare un elemento di un blocco che può cambiare dimensione si conserva il suo indice, non il suo indirizzo.

Errori Comuni da Evitare

1

Contare su malloc per avere zeri nei contatori.

2

Moltiplicare gli argomenti di calloc invece di passarli separati.

3

Scrivere v = realloc(v, ...) e perdere il blocco se fallisce.

4

Aggiornare la dimensione prima di sapere se realloc è riuscita.

5

Usare il puntatore vecchio dopo realloc.

6

Contare sugli zeri nella parte aggiunta da realloc.

7

Chiamare realloc(p, 0) invece di free.

8

Tenere puntatori a elementi di un blocco che cresce.

Riepilogo Concetti

Situazione	Tecnica	Note
Contatori e flag	calloc(n, sizeof *p)	tutti i byte a zero
Matrice vuota	calloc(righe * colonne, ...)	un solo blocco
Cambiare dimensione	tmp = realloc(p, byte)	il contenuto resta
realloc fallisce	tmp == NULL	p ancora valido
Ridurre ai dati usati	realloc dopo la lettura	n == 0: free
Togliere valori errati	filtro sul posto, poi realloc	due indici
Colonna in più	spostare dall'ultimo	r e c decrescenti
Elenco che parte vuoto	realloc(NULL, ...)	come malloc
Crescita di uno alla volta	costo come n^2	vedi 03-array-dinamici
Ricordare un elemento	indice	i puntatori si invalidano

SEZIONE 1

Esempi Pratici: calloc

Concetti base — tre file su blocchi azzerati.

1 – 010-calloc-azzera.c

Problema

due blocchi di 6 int.

Tecnica

calloc a confronto con malloc più azzeramento.

2 – 020-contatori-con-calloc.c

Problema

le lettere più frequenti di un testo cifrato.

Tecnica

26 contatori che partono da zero.

3 – 030-matrice-azzerata.c

Problema

il torneo di scacchi tutti contro tutti.

Tecnica

tabella dei punti in un blocco azzerato.

SEZIONE 1

Esempi Pratici: Problemi su calloc

040 – Istogramma dei voti

Un contatore per voto, nella scala scelta dal docente.

050 – Presenze del mese

Matrice giorni × studenti delle assenze.

060 – Parole per lunghezza

Leggibilità di un testo: parole per numero di lettere.

070 – Posti a teatro

Sala con i codici delle prenotazioni, 0 per i posti liberi.

080 – Crivello di Eratostene

Flag dei numeri composti fino a un milione.

SEZIONE 2

Esempi Pratici: realloc

Concetti base — tre file su ingrandire, fallire e ridurre.

1 – 010-ingrandire-un-blocco.c

Problema

le temperature di due settimane.

Tecnica

contenuto conservato, blocco spostato.

2 – 020-puntatore-temporaneo.c

Problema

una richiesta di memoria impossibile.

Tecnica

tmp = realloc, il blocco vecchio resta valido.

3 – 030-rimpicciolire.c

Problema

i punteggi della gara di matematica.

Tecnica

da 200 posti a quelli usati.

SEZIONE 2

Esempi Pratici: Problemi su realloc

040 – Posti aggiuntivi

Il corso di fotografia con posti aggiunti e tolti.

050 – Testo che si allunga

Il verbale del consiglio di classe, una frase alla volta.

060 – Misure ridotte

Letture errate del pluviometro tolte sul posto.

070 – Nuova materia

Una colonna in più nel tabellone dei voti.

080 – Fusione di classi

Due elenchi ordinati fusi dal fondo.

SEZIONE 3

Esempi Pratici: realloc in Pratica

Concetti base — tre file su NULL, costi e indici.

1 – 010-realloc-casi-limite.c

Problema

i libri consigliati per l'estate.

Tecnica

realloc(NULL, ...) e svuotamento con free.

2 – 020-quanto-costa-crescere.c

Problema

blocchi che crescono di 1 e di 16.

Tecnica

realloc e copie del caso peggiore.

3 – 030-puntatori-invalidati.c

Problema

il capoclasse dopo due nuovi arrivi.

Tecnica

indice invece di puntatore.

SEZIONE 3

Esempi Pratici: Problemi su realloc in Pratica

040 – Riga di lunghezza qualsiasi

Risposte del questionario lette in un blocco che cresce.

050 – Elenco di parole

Parole distinte di un tema e ripetizioni.

060 – Prenotazioni con indici

Postazioni che aumentano, prenotazioni sempre valide.

070 – Registro che cresce

Studenti che arrivano e che si trasferiscono.

080 – Archivio dei sensori

Un blocco di letture per sensore, ridimensionato con il periodo.